TECNOLOGO EN ANALISIS Y DESARROLLO DE SOFTWARE

SERVICIO NACIONAL DE APRENDIZAJE CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS

MANUAL TÉCNICO DE LA PLATAFORMA

PROYECTO: DRIVIQUE

INTEGRANTES:

LAURA VANESSA PEREZ PERDOMO
DANNA VALENTINA BARRIOS PENAGOS
EMILY SHARITH AMEZQUITA SAAVEDRA

FICHA:3145555

2026

Tabla de contenido

1. Información General del Proyecto y Ficha Técnica 4
1.1 Ficha Técnica del Sistema Drivique 4
1.2 Objetivos y Alcance del Sistema 4
1.3 Matriz de Roles de Desarrollo y Responsabilidades 4
2. Arquitectura de Software, Topología y Despliegue 6
2.1 Modelo Arquitectónico Cliente-Servidor Desacoplado 6
2.2 Topología de Red e Infraestructura en Hostinger VPS 6
2.3 Servidor Web y Proxy Reverso Seguro (Caddy 2) 7
2.4 Patrones de Diseño de Software Implementados 7
2.5 Justificación de la Migración Arquitectónica a VPS Hostinger 8
3. Stack Tecnológico Detallado y Estandarización 9
3.1 Backend Microservicio (Spring Boot) 9
3.2 Frontend Web SPA (React 19) 9
3.3 Frontend Móvil App (Expo / React Native) 10
3.4 Base de Datos y Persistencia 10
3.5 Matriz de Direccionamiento IP y Puertos de Producción 10
4. Estructura Exhaustiva de Directorios por Repositorio 11
4.1 Repositorio Back-end (Spring Boot 4.0.8) 11
4.2 Repositorio front-end_web (React 19 SPA) 11
4.3 Repositorio front-end_movil (React Native / Expo) 12
4.4 Repositorio database (PostgreSQL & Liquibase) 12
5. Modelo de Base de Datos y Diccionario de Datos Completo 13
5.1 Esquema `iam` (Identity and Access Management) 13
5.2 Esquema `fleet` (Flota Vehicular y Mantenimiento) 13
5.3 Esquema `catalog` (Catálogo Comercial y Tarifas) 14
5.4 Esquema `rental` (Proceso Transaccional de Reservas) 14
5.5 Esquema `billing` (Pagos y Facturación) 14
6. Catálogo Completo de Endpoints de la API REST 16
6.1 Endpoints de Autenticación y Cuentas de Usuario 16
6.2 Endpoints de Catálogo y Consulta de Vehículos 16

6.3 Endpoints de Reservas y Alquileres 16
6.4 Endpoints de Pagos y Webhook de Wompi 16
6.5 Endpoints de Flota y Mantenimiento 17
7. Seguridad, Autenticación y Control de Acceso (RBAC) 18
7.1 Arquitectura de Autenticación Stateless con JWT 18
7.2 Hashing Criptográfico de Contraseñas (BCrypt) 18
7.3 Matriz de Permisos por Rol (RBAC) 18
8. Integraciones de Servicios de Terceros 19
8.1 Pasarela de Pagos Wompi (Bancolombia) 19
8.2 Frankfurter API (Tasas de Cambio de Divisas) 19
8.3 Generación de Códigos QR (ZXing) 19
9. Guía de Instalación del Entorno de Desarrollo Local 20
9.1 Prerrequisitos de Software 20
9.2 Paso a Paso de Ejecución Local 20
10. Operaciones, Despliegue y Mantenimiento en Producción 21
10.1 Procedimiento de Despliegue Automatizado (deploy.sh) 21
10.2 Política y Automatización de Respaldos (backup.sh) 21
11. Aseguramiento de Calidad, Pruebas y Remediación 22
11.1 Cobertura de Código con JaCoCo (85%) 22
11.2 Pruebas de Humo (Smoke Tests) en Producción 22
11.3 Remediación y Cumplimiento de la Auditoría SENA 22

1. Información General del Proyecto y Ficha Técnica

El presente Manual Técnico ha sido elaborado como instrumento rector para el equipo de desarrollo, auditores de software y evaluadores del Servicio Nacional de Aprendizaje (SENA). En él se documenta de forma exhaustiva la arquitectura, la pila de tecnologías, las convenciones de codificación, el diccionario de datos de los 10 esquemas relacionales, el catálogo de endpoints de la API REST y los protocolos de despliegue y mantenimiento de la plataforma Drivique.

1.1 Ficha Técnica del Sistema Drivique

Parámetro	Detalle Técnico y Operativo
Nombre del Sistema	Drivique Platform (Sistema Integral de Gestión de Flotas y Renta de Vehículos)
Versión Oficial	Release 1.0.0 (Remediación E2E - ADSO 3145555)
Fecha de Certificación	Noviembre 2026
Tipo de Solución	Plataforma Web SPA Responsiva + Aplicación Móvil Híbrida / PWA + API REST Microservicio
Ambiente de Producción	Hostinger Cloud VPS dedicado (Ubuntu 24.04 LTS, IP pública: 2.25.75.54)
Dominios Oficiales	Web: https://drivique.tech | Móvil: https://app.drivique.tech | API: /api/v1
Licencia y Propiedad	Propiedad Intelectual SENA - Proyecto Formativo ADSO 3145555
Organización de Código	5 Repositorios Git independientes en GitHub bajo control de versiones estricto

1.2 Objetivos y Alcance del Sistema

Drivique es una solución empresarial diseñada para digitalizar de punta a punta la operación comercial y logística de agencias de arrendamiento vehicular en Colombia. El sistema cubre los siguientes frentes funcionales:

- **Exploración y Búsqueda:** Catálogo comercial en línea con filtros multidimensionales (categoría, marca, transmisión, combustible, rango de precio).
- **Motor de Tarifas:** Gestión de cotización automática con cálculo de días, depósitos de garantía y conversión de divisas en tiempo real.
- **Pagos Digitales:** Checkout seguro integrado con la pasarela Bancolombia Wompi y confirmación de pago asíncrona mediante Webhooks con firma criptográfica.
- **Contratación Legal:** Generación de contratos de arrendamiento en PDF con firma digital, marcas de agua legales y código QR dinámico para verificación vial.
- **Gestión de Flota:** Control de odómetro, nivel de combustible, inspecciones fotográficas de entrega y recepción, y alertas de mantenimiento preventivo.
- **Seguridad y Auditoría:** Módulos de auditoría forense inmutable, matriz de permisos por roles (RBAC) y soporte de incidentes en carretera.

1.3 Matriz de Roles de Desarrollo y Responsabilidades

Integrante / Desarrolladora	Rol Principal en el Proyecto	Repositorios y Módulos Transversales Desarrollados

Integrante / Desarrolladora	Rol Principal en el Proyecto	Repositorios y Módulos Transversales Desarrollados
Laura Vanessa Pérez Perdomo	Desarrolladora Full-Stack / Analista y Diseñadora de Software	Desarrollo transversal en los 5 repositorios (Back-end, front-end_web, front-end_movil, database, Documentation): arquitectura Spring Boot, persistencia JPA y PostgreSQL/Liquibase, seguridad JWT/RBAC, paneles administrativos General (/admin) y de Sucursal (/encargado), app móvil Expo/React Native e integración Wompi.
Danna Valentina Barrios Penagos	Desarrolladora Full-Stack / Analista y Diseñadora de Software	Desarrollo transversal en los 5 repositorios (Back-end, front-end_web, front-end_movil, database, Documentation): arquitectura React 19 y Spring Boot, modelado de base de datos PostgreSQL/Liquibase, paneles administrativos General (/admin) y de Sucursal (/encargado), catálogo interactivo, app móvil Expo/React Native y documentación técnica IEEE/SENA.
Emily Sharith Amézquita Saavedra	Desarrolladora Full-Stack / Analista y Diseñadora de Software	Desarrollo transversal en los 5 repositorios (Back-end, front-end_web, front-end_movil, database, Documentation): desarrollo móvil React Native / Expo SDK, servicios REST en Spring Boot, componentes web React 19, esquemas relacionales PostgreSQL/Liquibase, aseguramiento de calidad (QA) y documentación técnica IEEE/SENA.

2. Arquitectura de Software, Topología y Despliegue

Drivique adopta una Arquitectura Cliente-Servidor Desacoplada y Contenerizada mediante microservicios orquestados con Docker Compose. Toda la infraestructura se encuentra aprovisionada en un servidor Hostinger Cloud VPS con recursos dedicados, aislamiento en red privada Docker Bridge y proxy reverso con terminación SSL/TLS nativa.

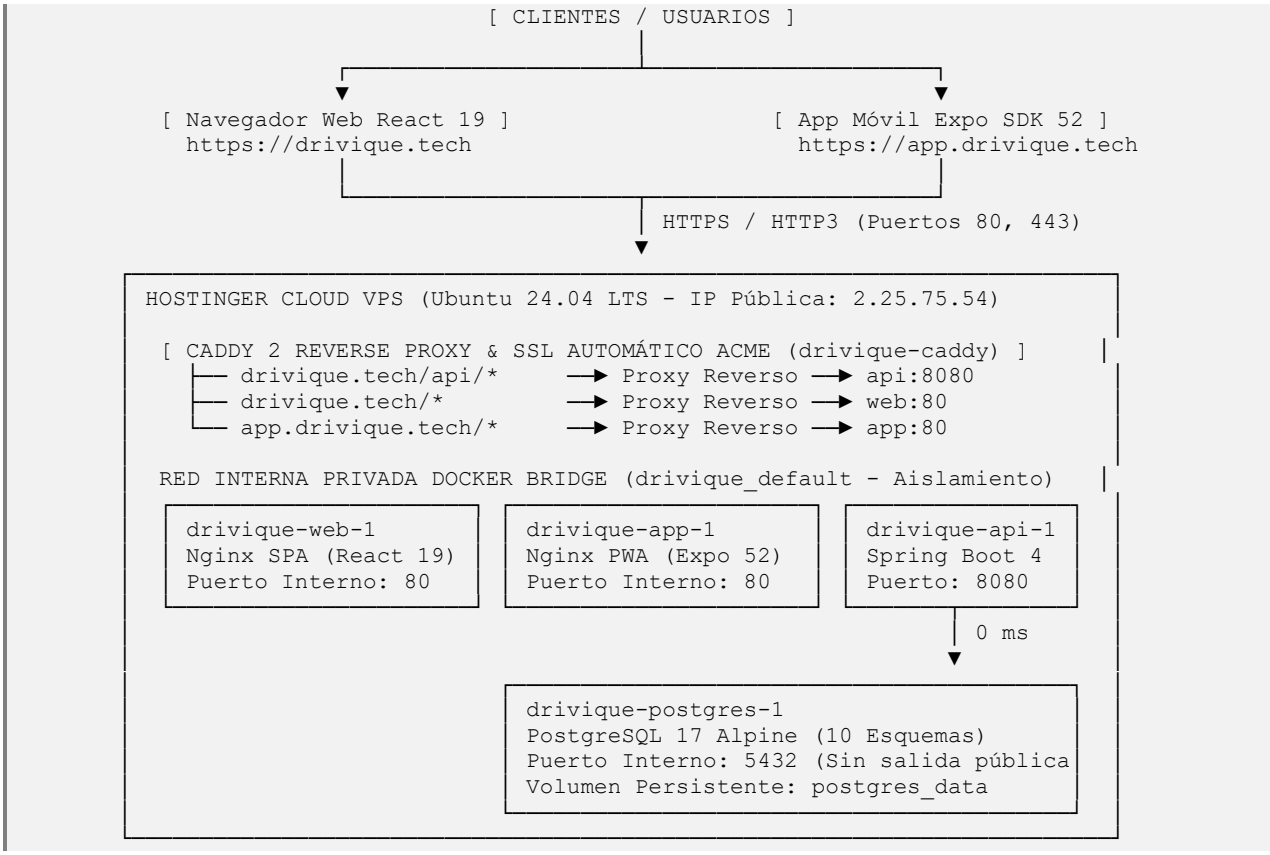
2.1 Modelo Arquitectónico Cliente-Servidor Desacoplado

El desacoplamiento estricto entre el Backend y las interfaces de usuario asegura que la lógica de negocio, validaciones transaccionales y persistencia permanezcan completamente centralizadas y protegidas:

- **Backend API REST Stateless:** El Backend API actúa como la única autoridad sobre los datos y las reglas del negocio. No procesa vistas HTML ni mantiene sesiones en servidor; se comunica exclusivamente mediante JSON sobre HTTPS.
- **Frontends Ligeros:** Tanto la aplicación Web (React 19) como la aplicación Móvil (Expo 52) son clientes independientes que gestionan su propio estado local en memoria (Zustand) y delegan la persistencia en el backend.
- **Capa de Persistencia Aislada:** El motor relacional PostgreSQL 17 mantiene aislamiento absoluto y cero exposición a la red pública, garantizando cumplimiento de estándares de seguridad OWASP.

2.2 Topología de Red e Infraestructura en Hostinger VPS

El siguiente diagrama esquematiza la topología de red y la arquitectura contenerizada desplegada en producción sobre el servidor Hostinger Cloud VPS, detallando el flujo de tráfico HTTPS/HTTP3, la terminación segura de certificados SSL mediante Caddy 2 y el aislamiento de microservicios y persistencia en la red privada Docker Bridge:



2.3 Servidor Web y Proxy Reverso Seguro (Caddy 2)

Caddy 2 cumple una triple función esencial en la arquitectura de producción de Drivique:

- **Terminación SSL/TLS Nativa:** Aprovisiona y renueva automáticamente certificados criptográficos TLS X.509 mediante Let's Encrypt sin requerir scripts adicionales de Certbot ni cortes de servicio.
- **Enrutador Inteligente:** Enruta el tráfico entrante de acuerdo con el dominio y la URI solicitada: /api/* hacia el backend Spring Boot (puerto 8080), drivique.tech hacia el contenedor Web (puerto 80) y app.drivique.tech hacia el contenedor Móvil (puerto 80).
- **Seguridad Perimetral:** Aplica compresión de alto rendimiento (zstd y gzip), impone políticas HSTS (Strict-Transport-Security: 31536000), oculta el header Server y restringe el tamaño máximo de cuerpo a 25 MB.

2.4 Patrones de Diseño de Software Implementados

Patrón de Diseño	Capa de Aplicación	Propósito y Beneficio Arquitectónico
MVC (Model-View-Controller)	Backend Spring Boot	Separa el manejo de peticiones HTTP (Controller), de las entidades (Model) y los DTOs de presentación.
Data Transfer Object (DTO)	Backend y Frontends	Desacopla las entidades internas de base de datos de los payloads expuestos en la red, evitando fugas de datos sensibles.

Patrón de Diseño	Capa de Aplicación	Propósito y Beneficio Arquitectónico
Repository Pattern	Backend (Spring Data JPA)	Abstrae el acceso a datos mediante interfaces fuertemente tipadas y soporte dinámico de especificaciones.
Facade Pattern	Frontends (Services / Axios)	Centraliza las peticiones HTTP y el manejo unificado de errores e inyección de tokens en un único punto.
Singleton	Spring Container / Zustand	Garantiza una única instancia de servicios de negocio en Spring y un único store de autenticación en los clientes.
Strategy Pattern	Backend (Pasarelas de Pago)	Permite intercambiar o incorporar nuevas pasarelas de pago (Wompi, MercadoPago) implementando una interfaz común.

2.5 Justificación de la Migración Arquitectónica a VPS Hostinger

Durante las etapas de prototipado se evaluaron servicios PaaS gratuitos (Render, Netlify y Supabase). No obstante, para garantizar calidad comercial y sustentación técnica, se ejecutó la migración completa a infraestructura dedicada en Hostinger Cloud VPS, sustentada en las siguientes ventajas comprobadas:

- **Eliminación de Cold Starts (Arranque en Frío):** En los servicios gratuitos, la API entraba en hibernación tras 15 minutos de inactividad, provocando demoras críticas de entre 50 y 90 segundos al iniciar la reserva. En VPS dedicado, la API responde en milisegundos las 24 horas del día.
- **Latencia Cero en Persistencia:** Al alojar PostgreSQL 17 Alpine en el mismo servidor bajo la red privada Docker Bridge, las transacciones entre Spring Boot y PostgreSQL operan con 0 ms de latencia física.
- **Unificación de Marca:** Se eliminaron las URLs provisionales dispersas (.netlify.app, .onrender.com), consolidando la marca bajo el dominio institucional drivique.tech y erradicando errores de CORS.
- **Seguridad y Control Total de Respaldos:** La base de datos no está expuesta en internet y se ejecutan copias de seguridad automatizadas diarias a las 02:30 AM con retención de 7 días mediante backup.sh.

3. Stack Tecnológico Detallado y Estandarización

Todas las librerías, motores y compiladores han sido estandarizados en versiones compatibles y estables para garantizar builds reproducibles y evitar fallas en tiempo de ejecución.

3.1 Backend Microservicio (Spring Boot)

Tecnología / Dependencia	Versión Exacta	Función y Justificación en el Sistema
Java Platform (JDK)	Java 21 LTS	Compilador y runtime moderno con soporte para Virtual Threads (Loom) y Records nativos.
Spring Boot Core	4.0.8	Framework base con auto-configuración de Spring WebMVC y contenedor embebido.
Spring Security	7.x (Core 4.0.8)	Seguridad de endpoints, filtros de autenticación JWT y configuración CORS.
JJWT (Java JWT)	0.12.6	Generación, firmado HMAC-SHA256 y verificación de tokens de acceso Bearer.
Spring Data JPA / Hibernate	6.x (Jakarta)	ORM relacional con mapeo de entidades sobre los 10 esquemas temáticos.
HikariCP	5.1.0	Pool de conexiones ultraligero y de alta velocidad hacia PostgreSQL.
Caffeine Cache	3.1.8	Caché de alto rendimiento en memoria RAM para catálogo y parámetros del sistema.
OpenPDF	1.3.40	Motor de renderizado de contratos legales de arrendamiento en formato PDF vectorizado.
Apache POI	5.4.0 (ooxml)	Generación de reportes y balances gerenciales en hojas de cálculo Excel (.xlsx).
ZXing (Zebra Crossing)	3.5.3 (javase)	Generación de códigos QR criptográficos incrustados en los contratos.
Bouncy Castle	1.78 (bcpkix)	Criptografía avanzada y soporte PKCS para firmas digitales.
SpringDoc OpenAPI	2.8.5	Documentación interactiva de la API REST accesible en /swagger-ui.html.
JaCoCo Maven Plugin	0.8.12	Monitoreo de cobertura de código con fallo de build si la cobertura es menor a 85%.
TestContainers	1.20.4	Contenedores Docker efímeros de PostgreSQL 17 para pruebas de integración reales.

3.2 Frontend Web SPA (React 19)

Tecnología / Dependencia	Versión Exacta	Función y Justificación en el Sistema
React Core	^19.2.5	Librería líder para interfaces interactivas con React Compiler y concurrencia.
Vite Bundler	^8.0.10	Compilador y empaquetador ultrarrápido con Hot Module Replacement (HMR).
TailwindCSS	^4.2.4	Framework de estilos atómicos con diseño responsivo mobile-first.
Zustand	^5.0.13	Manejador de estado global atómico (authStore, cartStore) sin boilerplate.
TanStack Query (React Query)	^5.100.9	Caché de datos de servidor, reintentos automáticos e invalidación de consultas.
Axios	^1.16.0	Cliente HTTP con interceptores de Bearer Token y manejo global de errores.

Tecnología / Dependencia	Versión Exacta	Función y Justificación en el Sistema
React Router DOM	^7.15.0	Enrutador declarativo con guardias de protección RoleBasedRoute.
i18next	^26.3.1	Internacionalización de interfaz para español e inglés en tiempo real.
Lucide React	0.475.0	Iconografía moderna SVG optimizada para componentes React.

3.3 Frontend Móvil App (Expo / React Native)

Tecnología / Dependencia	Versión Exacta	Función y Justificación en el Sistema
Expo SDK	~57.0.21	Plataforma de compilación y servicios nativos para iOS y Android.
React Native	0.86.3	Motor de componentes nativos móviles.
Expo Router	~57.0.20	Enrutamiento basado en archivos bajo la jerarquía de carpetas /app.
Expo Secure Store	~57.0.0	Almacenamiento seguro y cifrado en el Keychain de iOS y Keystore de Android.
React Native WebView	13.16.1	Contenedor web aislado para renderizar el checkout de la pasarela Wompi.
Zustand	^5.0.13	Manejo de estado global sincronizado para la sesión del conductor/cliente.

3.4 Base de Datos y Persistencia

Componente de Datos	Versión / Detalle	Función y Características
Motor Relacional	PostgreSQL 17 Alpine	Motor de base de datos relacional ACID optimizado en imagen Alpine Linux.
Gestor de Migraciones	Liquibase 4.x Core	Versionamiento de esquemas mediante master changelog y 41 releases YAML/SQL.
Extensiones Criptográficas	pgcrypto	Funciones nativas para UUIDs v4 (gen_random_uuid) y hashing en BD.
Validación Hibernate	validate	Verificación estricta de coherencia entre entidades Java y tablas de PostgreSQL.

3.5 Matriz de Direccionamiento IP y Puertos de Producción

Servicio / Contenedor	Nombre Docker	Puerto Externo	Puerto Interno	Acceso de Red
Proxy Reverso Caddy 2	drivique-caddy	80 (HTTP) / 443 (HTTPS)	80 / 443	Público (Internet)
Frontend Web (React 19)	drivique-web-1	Ninguno	80 (TCP)	Privado (Solo Caddy)
Frontend Móvil (Expo 52)	drivique-app-1	Ninguno	80 (TCP)	Privado (Solo Caddy)
Backend API (Spring Boot)	drivique-api-1	Ninguno	8080 (TCP)	Privado (Solo Caddy)
Base de Datos (PostgreSQL)	drivique-postgres-1	Ninguno	5432 (TCP)	Privado (Solo API)

4. Estructura Exhaustiva de Directorios por Repositorio

El código fuente del ecosistema Drivique está distribuido en 5 repositorios oficiales desacoplados. A continuación se desglosa la jerarquía detallada de carpetas y responsabilidades de cada uno:

4.1 Repositorio Back-end (Spring Boot 4.0.8)

Directorio raíz del código: src/main/java/com/drivique/api/

- **config/:** Clases de configuración de beans de Spring (Swagger/OpenAPI, WebMvcConfigurer, Beans de Caching y ObjectMapper).
- **controller/:** Controladores REST que publican los endpoints HTTP (@RestController). Mapean DTOs y delegan la lógica en los servicios.
- **dto/:** Records y clases de transferencia de datos para peticiones y respuestas (evitan exponer las entidades de base de datos).
- **exception/:** Manejador global de excepciones (@ControllerAdvice) que intercepta errores y construye respuestas JSON uniformes.
- **integration/:** Clientes HTTP que encapsulan la comunicación con Wompi, la API de Frankfurter y el motor SMTP de correo.
- **job/:** Tareas periódicas automatizadas (@Scheduled) que cancelan reservas expiradas y sincronizan tasas de divisas.
- **mapper/:** Interfaces MapStruct que ejecutan conversiones eficientes en tiempo de compilación entre Modelos y DTOs.
- **model/:** Entidades JPA (@Entity) con anotaciones @Table especificando el esquema correspondiente (ej. @Table(schema = "fleet")).
- **repository/:** Interfaces Spring Data JPA que extienden JpaRepository y JpaSpecificationExecutor para consultas atómicas y filtradas.
- **security/:** Filtros de interceptación JwtAuthenticationFilter, servicio UserDetailsServiceImpl y configuración de seguridad SecurityConfig.
- **service/:** Interfaces e implementaciones que concentran la lógica de negocio, validaciones transaccionales (@Transactional) y eventos.
- **specification/:** Criterios de búsqueda y filtros combinables para consultas avanzadas en el catálogo de vehículos.

4.2 Repositorio front-end_web (React 19 SPA)

Directorio raíz del código: web-drivique/src/

- **assets/:** Imágenes, logotipos vectoriales, favicon y recursos estáticos empaquetados por Vite.
- **components/:** Componentes de interfaz genéricos y reutilizables (Button, Input, Modal, DataTable, Navbar, Footer, Badge).

- **hooks/:** Hooks de React personalizados para aislar lógica de negocio (useAuth, useVehicles, usePagination).
- **i18n/:** Archivos JSON de traducción para el soporte multilingüe en español e inglés.
- **modules/:** Arquitectura Feature-Sliced. Contiene las vistas y subcomponentes aislados por módulo de negocio: admin/, auth/, catalog/, fleet/, rentals/, users/.
- **routes/:** Definición de rutas con React Router DOM y envolturas RoleBasedRoute para protección según el rol del usuario.
- **services/:** Capa de comunicación HTTP con Axios. Define funciones asíncronas para cada recurso del backend.
- **store/:** Almacenes de estado global en memoria utilizando Zustand (authStore, reservationStore, filterStore).
- **utils/:** Utilidades de formateo de moneda colombiana (COP), cálculos de fechas con date-fns y validadores de formulario.

4.3 Repositorio front-end_movil (React Native / Expo)

Directorio raíz del código: app-drivique/

- **app/:** Enrutador basado en archivos (File-based routing) de Expo Router. Las carpetas (auth), (tabs), checkout/ definen la navegación de la app.
- **assets/:** Recursos nativos (íconos adaptativos, pantallas de carga splash-screen).
- **components/:** Componentes táctiles nativos para dispositivos móviles (TouchButtons, Cards, SheetModals).
- **constants/:** Definición de colores de la paleta institucional, tipografías y espaciados de diseño móvil.
- **services/:** Funciones de consumo de API REST configuradas con Axios para móvil.
- **store/:** Estado global sincronizado en el dispositivo móvil con Zustand.

4.4 Repositorio database (PostgreSQL & Liquibase)

- **docker/:** Definición de Docker Compose para inicialización de la base de datos de desarrollo y scripts initdb.
- **changelog/db.changelog-master.yaml:** Archivo central que orquesta la ejecución secuencial de todos los changelogs en orden estricto de dependencias.
- **changelog/01_ddl/:** Definición de tablas iniciales DDL de los 10 esquemas temáticos.
- **changelog/02_dml/:** Datos semilla DML para usuarios administradores, marcas de vehículos y configuraciones iniciales.
- **changelog/releases/:** Releases incrementales versionados (hu-bd-03 a hu-bd-41) que garantizan que la BD se actualice sin pérdida de datos.

5. Modelo de Base de Datos y Diccionario de Datos Completo

Drivique implementa un modelo de datos de grado empresarial dividido en 10 esquemas relacionales temáticos. Esta separación previene el acoplamiento de tablas y garantiza el principio de mínimo privilegio en el acceso a datos.

5.1 Esquema `iam` (Identity and Access Management)

Controla la identidad, credenciales, roles y tokens de actualización de los usuarios de la plataforma.

Tabla	Columna Clave	Tipo de Dato	Restricción	Descripción Funcional
iam.users	id	UUID	PRIMARY KEY	Identificador único universal del usuario.
iam.users	email	VARCHAR(150)	UNIQUE, NOT NULL	Correo electrónico institucional o personal.
iam.users	password_hash	VARCHAR(255)	NOT NULL	Hash BCrypt con factor de costo 12.
iam.users	is_active	BOOLEAN	DEFAULT TRUE	Estado de habilitación del usuario en la plataforma.
iam.roles	id	SERIAL	PRIMARY KEY	Identificador autonumérico del rol.
iam.roles	name	VARCHAR(50)	UNIQUE, NOT NULL	Nombre del rol (SUPER_ADMIN, ADMIN_AGENCY, etc.).
iam.user_roles	user_id, role_id	UUID, INT	COMPOSITE PK, FK	Relación muchos a muchos entre usuarios y roles.
iam.refresh_tokens	token	VARCHAR(255)	UNIQUE, NOT NULL	Token criptográfico opaco para renovación de JWT.

5.2 Esquema `fleet` (Flota Vehicular y Mantenimiento)

Almacena la información técnica, odómetro, placas y estado operativo del inventario físico de vehículos.

Tabla	Columna Clave	Tipo de Dato	Restricción	Descripción Funcional
fleet.vehicles	id	UUID	PRIMARY KEY	Identificador único de la unidad física del vehículo.
fleet.vehicles	license_plate	VARCHAR(10)	UNIQUE, NOT NULL	Placa oficial de tránsito colombiano (ej. ABC-123).
fleet.vehicles	model_id	INT	FOREIGN KEY	Referencia al modelo comercial en catalog.models.
fleet.vehicles	branch_id	INT	FOREIGN KEY	Sucursal física donde se encuentra estacionado.
fleet.vehicles	mileage	INT	NOT NULL	Kilometraje actual acumulado del odómetro.
fleet.vehicles	status_id	INT	FOREIGN KEY	Estado actual (DISPONIBLE, ALQUILADO, MANTENIMIENTO).
fleet.maintenance_logs	id	UUID	PRIMARY KEY	Registro de orden de servicio mecánico preventivo.
fleet.inspections	id	UUID	PRIMARY KEY	Inspección fotográfica de check-in o check-out.

5.3 Esquema `catalog` (Catálogo Comercial y Tarifas)

Define los vehículos visibles al cliente final, sus fotografías, especificaciones y tablas de precios.

Tabla	Columna Clave	Tipo de Dato	Restricción	Descripción Funcional
catalog.brands	id	SERIAL	PRIMARY KEY	Marca automotriz (Toyota, Chevrolet, Mazda, Renault).
catalog.models	id	SERIAL	PRIMARY KEY	Línea o modelo comercial (Corolla, Onix, CX-5, Duster).
catalog.categories	id	SERIAL	PRIMARY KEY	Segmento (Económico, SUV, Sedán, Pickup, Eléctrico).
catalog.daily_rates	id	SERIAL	PRIMARY KEY	Tarifa diaria base en pesos colombianos (COP).
catalog.daily_rates	deposit_amount	DECIMAL(12,2)	NOT NULL	Monto requerido de depósito de garantía en tarjeta.

5.4 Esquema `rental` (Proceso Transaccional de Reservas)

Gestiona el ciclo de vida del alquiler desde la cotización hasta la devolución y cierre.

Tabla	Columna Clave	Tipo de Dato	Restricción	Descripción Funcional
rental.reservations	id	UUID	PRIMARY KEY	Identificador único de la reserva de alquiler.
rental.reservations	booking_reference	VARCHAR(20)	UNIQUE, NOT NULL	Código de reserva visible al usuario (DRV-RES-XXXX).
rental.reservations	customer_id	UUID	FOREIGN KEY	Referencia al cliente en iam.users.
rental.reservations	vehicle_id	UUID	FOREIGN KEY	Referencia al vehículo asignado en fleet.vehicles.
rental.reservations	pickup_datetime	TIMESTAMP	NOT NULL	Fecha y hora programada de entrega del vehículo.
rental.reservations	return_datetime	TIMESTAMP	NOT NULL	Fecha y hora programada de devolución en sucursal.
rental.reservations	total_price	DECIMAL(12,2)	NOT NULL	Monto total liquidado del alquiler con impuestos.
rental.reservations	status	VARCHAR(30)	NOT NULL	Estado: PENDING_PAYMENT, CONFIRMED, ACTIVE, COMPLETED, CANCELLED.

5.5 Esquema `billing` (Pagos y Facturación)

Registra las transacciones procesadas a través de la pasarela Wompi y las facturas electrónicas emitidas.

Tabla	Columna Clave	Tipo de Dato	Restricción	Descripción Funcional
billing.payments	id	UUID	PRIMARY KEY	Identificador interno del intento de pago.
billing.payments	reservation_id	UUID	FOREIGN KEY	Reserva a la cual se abona la transacción.

Tabla	Columna Clave	Tipo de Dato	Restricción	Descripción Funcional
billing.payments	wompi_transaction_id	VARCHAR(100)	UNIQUE	Identificador transaccional devuelto por Wompi.
billing.payments	amount_in_cents	BIGINT	NOT NULL	Monto exacto en centavos (ej. 15000000 = $150.000 COP).
billing.payments	payment_status	VARCHAR(30)	NOT NULL	APPROVED, DECLINED, VOIDED, ERROR.
billing.currency_rates	id	SERIAL	PRIMARY KEY	Histórico de tasas de cambio EUR/USD a COP.

6. Catálogo Completo de Endpoints de la API REST

Todos los endpoints del Backend están estandarizados bajo el prefijo canónico /api/v1/ y protegidos con políticas de seguridad según el rol de acceso.

6.1 Endpoints de Autenticación y Cuentas de Usuario

Método	Ruta de Endpoint	Acceso Requerido	Descripción y Payload
POST	/api/v1/auth/login	Público	Autentica credenciales y genera Access Token JWT + Refresh Token.
POST	/api/v1/auth/register	Público	Registra un nuevo cliente con hash BCrypt de contraseña.
POST	/api/v1/auth/refresh	Público (Refresh)	Renueva el Access Token expirado utilizando el Refresh Token válido.
GET	/api/v1/users/profile	Autenticado (Bearer)	Retorna el perfil, roles y datos del usuario en sesión.
PUT	/api/v1/users/profile	Autenticado (Bearer)	Actualiza datos personales, teléfono y dirección del usuario.

6.2 Endpoints de Catálogo y Consulta de Vehículos

Método	Ruta de Endpoint	Acceso Requerido	Descripción y Payload
GET	/api/v1/vehicles/search	Público	Búsqueda paginada con filtros por fechas, ciudad, marca y precio.
GET	/api/v1/vehicles/{id}	Público	Obtiene el detalle comercial de un vehículo y tarifas diarias.
GET	/api/v1/catalog/brands	Público	Lista todas las marcas registradas en el catálogo comercial.
GET	/api/v1/catalog/categories	Público	Lista los segmentos vehiculares (Económico, SUV, Sedán, etc.).

6.3 Endpoints de Reservas y Alquileres

Método	Ruta de Endpoint	Acceso Requerido	Descripción y Payload
POST	/api/v1/reservations	CUSTOMER / ADMIN	Crea una nueva reserva en estado PENDING_PAYMENT con bloqueo temporal.
GET	/api/v1/reservations/{id}	Propietario / ADMIN	Consulta el estado detallado de una reserva por su ID o referencia.
GET	/api/v1/reservations/my-bookings	CUSTOMER	Lista el histórico de reservas del cliente autenticado.
POST	/api/v1/reservations/{id}/cancel	Propietario / ADMIN	Cancela una reserva y libera el vehículo en fleet.vehicles.

6.4 Endpoints de Pagos y Webhook de Wompi

Método	Ruta de Endpoint	Acceso Requerido	Descripción y Payload
POST	/api/v1/payments/wompi/checkout	CUSTOMER / ADMIN	Inicializa la orden de pago y genera el hash de integridad para el widget.

Método	Ruta de Endpoint	Acceso Requerido	Descripción y Payload
POST	/api/v1/payments/wompi/webhook	Wompi (Público)	Recibe eventos de pago asíncronos y valida la firma SHA-256.
GET	/api/v1/invoices/{id}/pdf	Propietario / ADMIN	Descarga la factura electrónica generada en formato PDF.

6.5 Endpoints de Flota y Mantenimiento

Método	Ruta de Endpoint	Acceso Requerido	Descripción y Payload
GET	/api/v1/fleet/vehicles	OPERATOR / ADMIN	Consulta el inventario físico total con kilometraje y estado operativo.
POST	/api/v1/fleet/vehicles	ADMIN_AGENCY / SUPER	Registra una nueva unidad vehicular con su placa y modelo.
PUT	/api/v1/fleet/vehicles/{id}/mileage	OPERATOR	Actualiza el odómetro tras una inspección de recepción.
POST	/api/v1/fleet/inspections	OPERATOR	Registra la inspección fotográfica de entrega o devolución.

7. Seguridad, Autenticación y Control de Acceso (RBAC)

La seguridad de Drivique se estructura bajo el principio de defensa en profundidad, combinando autenticación sin estado (Stateless JWT), hashing criptográfico no reversible y aislamiento estricto por roles.

7.1 Arquitectura de Autenticación Stateless con JWT

Al autenticarse exitosamente en /api/v1/auth/login, el backend emite un JSON Web Token firmado criptográficamente con el algoritmo HMAC-SHA256 utilizando la clave secreta de 256 bits configurada en JWT_SECRET:

- **Access Token:** Contiene las credenciales del usuario, claims de roles y una expiración de 24 horas (86400000 ms). Se transporta en cada petición HTTP en la cabecera Authorization: Bearer <token>.
- **Refresh Token:** Token opaco de alta entropía persistido en iam.refresh_tokens que permite solicitar nuevos Access Tokens sin obligar al usuario a digitar nuevamente sus credenciales.

7.2 Hashing Criptográfico de Contraseñas (BCrypt)

Las contraseñas se almacenan protegidas mediante el algoritmo BCrypt con un factor de costo computacional de 12 (4096 iteraciones de sal y hash), mitigando ataques de fuerza bruta y diccionarios precomputados.

7.3 Matriz de Permisos por Rol (RBAC)

Módulo / Funcionalidad	CUSTOMER	OPERATOR	ADMIN_AGENCY	SUPER_ADMIN
Búsqueda en Catálogo y Cotización	Permitido	Permitido	Permitido	Permitido
Creación de Reservas y Pagos Wompi	Permitido	Permitido	Permitido	Permitido
Check-in / Check-out e Inspecciones	Denegado	Permitido	Permitido	Permitido
Gestión de Flota y Órdenes de Taller	Denegado	Denegado	Permitido	Permitido
Gestión de Sucursales y Tarifas	Denegado	Denegado	Permitido (Local)	Permitido (Total)
Auditoría Global y Configuración de Sistema	Denegado	Denegado	Denegado	Permitido

8. Integraciones de Servicios de Terceros

8.1 Pasarela de Pagos Wompi (Bancolombia)

La integración con Wompi permite procesar tarjetas de crédito, débito PSE, Nequi y transferencias Bancolombia:

- **Widget Checkout:** El cliente selecciona el método de pago; el frontend inicializa el widget oficial de Wompi pasando la referencia única de reserva (DRV-RES-XXXX), el monto en centavos y la firma de integridad calculada.
- **Validación de Webhook:** Tras procesar la transacción, los servidores de Wompi envían un webhook POST a https://drivique.tech/api/v1/payments/wompi/webhook. El backend calcula la firma SHA-256 utilizando WOMPI_EVENT_SECRET. Si la firma coincide, la reserva pasa a estado PAID_CONFIRMED.

8.2 Frankfurter API (Tasas de Cambio de Divisas)

El backend consulta periódicamente la API pública de Frankfurter (https://api.frankfurter.dev/v1/latest) para obtener la tasa de cambio de Dólar (USD) y Euro (EUR) a Peso Colombiano (COP), facilitando cotizaciones para turistas extranjeros. Se implementa un timeout de 5000 ms y respaldo en base de datos.

8.3 Generación de Códigos QR (ZXing)

Cada contrato generado incorpora un código QR dinámico que contiene un enlace seguro hacia la verificación del contrato en la nube, permitiendo a las autoridades de tránsito validar la autenticidad del alquiler sin necesidad de documentos físicos.

9. Guía de Instalación del Entorno de Desarrollo Local

Para ejecutar la plataforma Drivique en un equipo local de desarrollo se deben seguir los siguientes pasos estandarizados:

9.1 Prerrequisitos de Software

- **Java:** Java Development Kit (JDK) 21 LTS instalado y configurado en la variable JAVA_HOME.
- **Node.js:** Node.js versión 22.x LTS o superior con gestor de paquetes npm o pnpm.
- **Docker:** Docker Desktop con Docker Engine 24+ y Docker Compose v2 activo.
- **Git:** Git 2.40+ para control de versiones.

9.2 Paso a Paso de Ejecución Local

1. Clonar los repositorios y levantar la base de datos:

```
# Iniciar el motor PostgreSQL en contenedor local
cd database/docker
docker compose up -d


# Ejecutar las migraciones de Liquibase para crear los 10 esquemas
mvn liquibase:update -Dliquibase.changelog=changelog/db.changelog-master.yaml
```

2. Iniciar el Backend Spring Boot:

```
cd Back-end
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

3. Iniciar el Frontend Web:

```
cd front-end_web/web-drivique
npm install
npm run dev
```

4. Iniciar la App Móvil con Expo:

```
cd front-end_movil/app-drivique
npm install
npx expo start
```

10. Operaciones, Despliegue y Mantenimiento en Producción

El mantenimiento del servidor Hostinger VPS se gestiona a través de scripts de shell estandarizados ubicados en /opt/drivique/deploy/scripts/.

10.1 Procedimiento de Despliegue Automatizado (deploy.sh)

Para actualizar la plataforma a la última versión estable de la rama main:

```
# Conectarse al servidor por SSH
ssh deploy@2.25.75.54


# Entrar al directorio de despliegue y ejecutar el script
cd /opt/drivique/deploy
./scripts/deploy.sh
```

El script realiza automáticamente:

- **1. Git Pull:** Actualización del código fuente mediante git checkout main y git pull.
- **2. Backup Preventivo:** Respaldo de seguridad preventivo de PostgreSQL antes de aplicar cambios.
- **3. Build Docker:** Compilación multi-stage de las imágenes Docker (dc build).
- **4. Migración Liquibase:** Ejecución del contenedor efímero de Liquibase para aplicar los changesets nuevos.
- **5. Up Contenedores:** Reinicio controlado de contenedores con cero tiempo de inactividad.
- **6. Healthcheck:** Monitoreo continuo del healthcheck del backend hasta certificar estado healthy.

10.2 Política y Automatización de Respaldos (backup.sh)

Se ejecutan copias de seguridad de la base de datos de forma automática todas las noches a las 02:30 AM vía crontab:

```
30 2 * * * /opt/drivique/deploy/scripts/backup.sh >> /opt/drivique/backups/backup.log 2>&1
```

El script realiza un dump lógico con compresión gzip nivel 9 (db-YYYYMMDD-HHMMSS.sql.gz) y elimina automáticamente los respaldos con más de 7 días de antigüedad.

11. Aseguramiento de Calidad, Pruebas y Remediación

11.1 Cobertura de Código con JaCoCo (85%)

El archivo pom.xml del Backend impone una regla estricta de compilación con jacoco-maven-plugin. Si la cobertura de pruebas unitarias desciende del 85% en las clases del core de negocio, la compilación de Maven falla de inmediato:

```
<rule>
  <element>PACKAGE</element>
  <limits>
    <limit>
      <counter>LINE</counter>
      <value>COVEREDRATIO</value>
      <minimum>0.85</minimum>
    </limit>
  </limits>
</rule>
```

11.2 Pruebas de Humo (Smoke Tests) en Producción

Posterior a cada despliegue se validan los siguientes endpoints en vivo:

- **Salud de la API:** GET https://drivique.tech/api/actuator/health retorna status UP.
- **Catálogo:** GET https://drivique.tech/api/v1/vehicles/search?size=1 retorna código 200 OK con datos de flota.
- **Seguridad SSL:** curl -Iv https://drivique.tech certifica protocolo HTTP/2 y terminación TLS 1.3 con Let's Encrypt.

11.3 Remediación y Cumplimiento de la Auditoría SENA

La versión Release 1.0.0 incorpora la remediación integral de las 23 Historias de Usuario auditadas en el informe de seguimiento de ADSO 3145555, certificando la estabilidad, completitud y cumplimiento normativo del software para su sustentación final.